Machine Translated by Google

CAPÍTULO 6

Sincronización

Herramientas



Ceremonias

6.12 El pseudocódigo de la Figura 6.15 ilustra las operaciones básicas push() y pop() de una pila basada en arreglos. Suponiendo que este algoritmo se pueda utilizar en un entorno concurrente, responda las siguientes preguntas:

a. ¿Qué datos presentan una condición de carrera?

b. ¿Cómo se podría solucionar la condición de carrera?

6.13 Las condiciones de carrera son posibles en muchos sistemas informáticos. Consideremos un sistema de subastas en línea donde se debe mantener la puja más alta actual para cada artículo. Una persona que desea pujar por un artículo llama a la función puja(cantidad) , que compara la cantidad ofrecida con la puja más alta actual. Si la cantidad supera la puja más alta actual, esta se establece en la nueva cantidad. Esto se ilustra a continuación:

```
oferta vacía(cantidad doble) {
    si (cantidad > oferta más alta)
        oferta más alta = cantidad;
}
```

Describe cómo es posible que se produzca una condición de carrera en esta situación y qué se podría hacer para evitar que ocurra.

6.14 El siguiente ejemplo de programa se puede utilizar para sumar los valores de un array de tamaño N elementos en paralelo en un sistema que contiene N núcleos de procesamiento (hay un procesador independiente para cada elemento del array):

```
para j = 1 hasta log 2(N)
    { para k = 1 hasta N
        { si ((k + 1) % pow(2,j) == 0) { valores[k]
            += valores[k - pow(2,(j-1))]
        }
    }
}
```

Machine Translated by Google

Esto tiene el efecto de sumar los elementos del arreglo como una serie de sumas parciales, como se muestra en la Figura 6.16. Una vez ejecutado el código, la suma de todos los elementos del arreglo se almacena en la última posición del arreglo. ¿Existen condiciones de carrera en el ejemplo de código anterior? En caso afirmativo, identifique dónde se producen e ilústrelas con un ejemplo. En caso negativo, demuestre por qué este algoritmo no presenta condiciones de carrera.

6.15 Las instrucciones compare y swap() se pueden usar para diseñar estructuras de datos sin bloqueo, como pilas, colas y listas. El ejemplo de programa que se muestra en la Figura 6.17 presenta una posible solución para una pila sin bloqueo usando instrucciones CAS , donde la pila se representa como una lista enlazada de elementos Node , con top representando la parte superior de la pila. ¿Esta implementación está libre de condiciones de carrera?

```
push(item) { acquire();
    if (top < SIZE)
    { stack[top] = item; top++;


    }
    demás
        ERROR
    release();
}

pop()
    { acquire(); if (!is
    empty()) { top--; item =
        stack[top];
        release(); return item;


    }
    demás
        ERROR
    release();
}

está vacío() { si (top
    == 0)
        Devuelve verdadero;
    de lo
        contrario, devuelve falso;
}
```

Figura 6.15 Pila basada en arreglos para el Ejercicio 6.12.

Machine Translated by Google

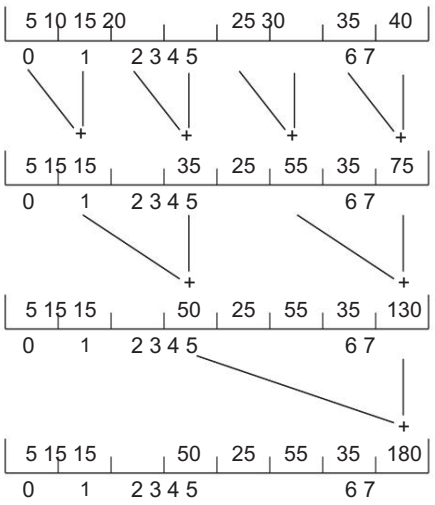


Figura 6.16 Suma de una matriz como una serie de sumas parciales para el Ejercicio 6.14.

6.16 Un enfoque para usar compare y swap() para implementar un spin-lock es el siguiente:

```
void lock spinlock(int *lock) {
    mientras (comparar e intercambiar(bloqueo, 0, 1) != 0)
        ; /* girar */
}
```

Un enfoque alternativo sugerido es utilizar el modismo "comparar y comparar e intercambiar" , que verifica el estado del bloqueo antes de invocar el Operación de comparación e intercambio() . (La razón detrás de este enfoque es para invocar compare y swap() solo si el bloqueo está disponible actualmente.)
Esta estrategia se muestra a continuación:

```
void lock spinlock(int *lock) {
{
    mientras (verdadero) {
        si (*bloqueo == 0) {
            /* Parece que hay un bloqueo disponible */

            si (!comparar y intercambiar(bloqueo, 0, 1))
                romper;
        }
    }
}
```

¿ Funciona adecuadamente esta expresión idiomática de "comparar y comparar e intercambiar"?
¿Para implementar spinlocks? Si es así, explique. Si no, ilustre cómo.
La integridad de la cerradura está comprometida.

Machine Translated by Google


```
typedef struct nodo { valor t
    datos; struct nodo
    *siguiente;
} Nodo;

Nodo *top; // parte superior de la pila

void push(valor t elemento) {
    Nodo *nodo antiguo;
    Nodo *nuevo nodo;

    nuevo nodo = malloc(sizeof(Nodo)); nuevo nodo-
    >datos = elemento;

    hacer
        { nodo antiguo =
        superior; nodo nuevo->siguiente = nodo antiguo;

    } mientras (comparar e intercambiar(top, nodo antiguo, nodo nuevo) != nodo antiguo);
}

valor t pop() {
    Nodo *nodo antiguo;
    Nodo *nuevo nodo;

    hacer
        { nodo antiguo =
        superior; si (nodo antiguo ==
            NULL) devolver NULL;
        nuevo nodo = nodo antiguo->siguiente;

    } mientras (comparar e intercambiar(top, nodo antiguo, nodo nuevo) != nodo antiguo);

    devolver nodo->datos antiguos;
}
```

Figura 6.17 Pila sin bloqueo para el ejercicio 6.15.

6.17 Algunas implementaciones de semáforos proporcionan una función getValue() que devuelve el valor actual de un semáforo. Esta función puede, por ejemplo, invocarse antes de llamar a wait() para que un proceso solo llame a wait().

Machine Translated by Google


```
mientras (verdadero)
    { flag[i] = verdadero;

    mientras (bandera[j]) {
        si (turno == j) {
            flag[i] = false; while
            (turn == j) ; /* no hacer
                nada */ flag[i] = true;

        }
    }

        /* sección crítica */

    turno = j;
    bandera[i] = falso;

        /* sección restante */
}
```

Figura 6.18 Estructura del proceso Pi en el algoritmo de Dekker.

Si el valor del semáforo es > 0, se evita el bloqueo mientras se espera el semáforo. Por ejemplo:

```
Si (getValue(&sem) > 0)
    esperar(&sem);
```

Muchos desarrolladores se oponen a dicha función y desaconsejan su uso. Describe un posible problema que podría ocurrir al usar la función getValue() en este escenario.

6.18 La primera solución de software correcta conocida para el problema de la sección crítica para dos procesos fue desarrollada por Dekker. Los dos procesos, P0 y P1, P1 ,comparten las siguientes variables:

```
bandera booleana[2]; /* inicialmente falsa */ turno entero;
```

La estructura del proceso Pi (i == 0 o 1) se muestra en la Figura 6.18. El otro proceso es Pj (j == 1 o 0). Demuestre que el algoritmo satisface los tres requisitos del problema de la sección crítica.

6.19 La primera solución de software correcta conocida para el problema de la sección crítica para n procesos con un límite inferior de espera de n − 1 turnos fue presentada por Eisenberg y McGuire. Los procesos comparten las siguientes variables:

Machine Translated by Google


```
mientras (verdadero)
    { mientras
        (verdadero) { flag[i] ≡
        querer entrar; j = girar;

        mientras (j != i) { si (flag[j] !=
            idle) { j = turn; else


                j = (j + 1) % n;
        }

        flag[i] = en cs; j = 0;


        mientras ( (j < n) && (j == i || flag[j] != in cs))
            j++;

        si ( (j >= n) && (turno == i || flag[turno] == inactivo))
            romper;
    }

        /* sección crítica */

    j = (giro + 1) % n;

    mientras (flag[j] == idle) j = (j + 1)
        % n;

    turno = j;
    bandera[i] = inactiva;

        /* sección restante */
}
```

Figura 6.19 Estructura del proceso Pi en el algoritmo de Eisenberg y McGuire.

```
enum pstate {idle, want in, in cs}; pstate flag[n];
int turn;
```

Todos los elementos de la bandera están inicialmente inactivos. El valor inicial del turno es irrelevante (entre 0 y n-1). La estructura del proceso Pi se muestra en la Figura 6.19. Demuestre que el algoritmo satisface los tres requisitos del problema de la sección crítica.

Machine Translated by Google

6.20 Explique por qué implementar primitivas de sincronización deshabilitando las interrupciones no es apropiado en un sistema de un solo procesador si las primitivas de sincronización se van a utilizar en programas de nivel de usuario.

6.21 Considere cómo implementar un bloqueo mutex utilizando las instrucciones `compare()` e `swap()` . Suponga que la siguiente estructura que define el bloqueo mutex está disponible:

```
typedef struct { int
        disponible; } bloqueo;
```

El valor (available == 0) indica que el bloqueo está disponible, y un valor de 1 indica que el bloqueo no está disponible. Utilizando esta estructura, ilustre cómo se pueden implementar las siguientes funciones usando las instrucciones compare y swap() :

- void acquire(lock *mutex)
- void release(lock *mutex)

Asegúrese de incluir cualquier inicialización que pueda ser necesaria.

6.22 Explique por qué las interrupciones no son apropiadas para implementar primitivas de sincronización en sistemas multiprocesador.

6.23 La implementación de bloqueos mutex descrita en la Sección 6.5 presenta problemas de espera activa. Describa qué cambios serían necesarios para que un proceso que espera adquirir un bloqueo mutex se bloquee y se coloque en una cola de espera hasta que el bloqueo esté disponible.

6.24 Suponga que un sistema tiene varios núcleos de procesamiento. Para cada uno de los siguientes escenarios, describa cuál es un mejor mecanismo de bloqueo: un spinlock o un mutex lock, donde los procesos en espera permanecen inactivos mientras esperan a que el bloqueo esté disponible:

- El candado debe mantenerse presionado durante un breve período de tiempo.
- El candado debe mantenerse presionado durante un período prolongado.
- Un hilo puede quedar inactivo mientras se mantiene presionado el candado.

6.25 Suponga que un cambio de contexto tarda T tiempo. Proponga un límite superior (en términos de T) para mantener un spinlock. Si el spinlock se mantiene durante más tiempo, un bloqueo mutex (donde los hilos en espera se ponen en estado de suspensión) es una mejor alternativa.

6.26 Un servidor web multihilo desea llevar un registro del número de solicitudes que atiende (conocido como hits). Considere las dos estrategias siguientes para evitar una condición de carrera en la variable hits. La primera estrategia consiste en utilizar un bloqueo mutex básico al actualizar hits:

```
aciertos de
enteros; bloqueo mutex, bloqueo de aciertos;

hit lock.acquire();
```

Machine Translated by Google


```
#define MAX PROCESSES 255 int número
de procesos = 0;

/* la implementación de fork() llama a esta función */ int allocate process() { int new pid;

    if (número de procesos == PROCESOS MÁXIMOS) return -1; else { /
        * asignar los
    recursos
        de proceso necesarios */ ++número de procesos;

        devolver nuevo pid;
    }
}

/* la implementación de exit() llama a esta función */ void release process() { /* libera los
recursos del proceso */ --número de
    procesos;
}
```

Figura 6.20 Procesos de asignación y liberación para el Ejercicio 6.27.

```
hits++; hit
lock.release();
```

Una segunda estrategia consiste en utilizar un número entero atómico:

```
atomic t hits; atomic
inc(&hits);
```

Explica cuál de estas dos estrategias es más eficiente.

6.27 Considere el ejemplo de código para asignar y liberar procesos que se muestra en la Figura 6.20.

a. Identifique la(s) condición(es) de

carrera. b. Suponga que tiene un bloqueo mutex llamado mutex con las operaciones acquire() y release(). Indique dónde debe colocarse el bloqueo para evitar la(s) condición(es) de carrera.

c. ¿Podríamos reemplazar la variable entera?

```
número entero de procesos = 0
```

con el entero atómico

```
Número atómico t de procesos = 0
```

Machine Translated by Google

¿Para prevenir la(s) condición(es) de carrera?

6.28 Los servidores pueden diseñarse para limitar el número de conexiones abiertas. Por ejemplo, un servidor puede querer tener solo N conexiones de socket en un momento dado. Tan pronto como se establezcan N conexiones, el servidor no aceptará otra conexión entrante hasta que se libere una conexión existente. Ilustre cómo un servidor puede usar semáforos para limitar el número de conexiones concurrentes.

6.29 En la Sección 6.7, utilizamos la siguiente ilustración como un uso incorrecto de semáforos para resolver el problema de la sección crítica:

```
esperar(mutex);
...
sección crítica
...
esperar(mutex);
```

Explique por qué esto es un ejemplo de fallo de transmisión en directo.

6.30 Demostrar que los monitores y los semáforos son equivalentes en la medida en que pueden utilizarse para implementar soluciones a los mismos tipos de problemas de sincronización.

6.31 Describa en qué se diferencia la operación signal() asociada con los monitores de la operación correspondiente definida para los semáforos.

6.32 Supongamos que la instrucción signal() solo puede aparecer como la última instrucción en una función de monitor. Sugiera cómo se puede simplificar la implementación descrita en la Sección 6.7 en esta situación.

6.33 Considere un sistema compuesto por los procesos P1 , P2 , ..., Pn , cada uno con un número de prioridad único. Escriba un monitor que asigne tres impresoras idénticas a estos procesos, utilizando los números de prioridad para determinar el orden de asignación.

6.34 Un archivo le debe ser compartido entre diferentes procesos, cada uno con un número único. Varios procesos pueden acceder al archivo le simultáneamente, sujeto a la siguiente restricción: la suma de todos los números únicos asociados a los procesos que acceden al archivo le debe ser menor que n. Escriba un monitor para coordinar el acceso al archivo le.

6.35 Cuando se realiza una señal sobre una condición dentro de un monitor, el proceso de señalización puede continuar su ejecución o transferir el control al proceso que recibe la señal. ¿En qué se diferenciaría la solución del ejercicio anterior con estas dos formas distintas de realizar la señalización?

6.36 Diseñe un algoritmo para un monitor que implemente un reloj despertador que permita a un programa que lo llama retrasarse durante un número determinado de unidades de tiempo (ticks). Puede asumir la existencia de un reloj de hardware real que invoca la función tick() en su monitor a intervalos regulares.

6.37 Analice las formas en que se podría abordar el problema de la inversión de prioridades en un sistema en tiempo real. Asimismo, analice si las soluciones podrían implementarse en el contexto de un planificador de reparto proporcional.

Machine Translated by Google